

Part 1. Software Security

Oliver Wright

2203984

1. Context

The PostgreSQL database server represents a crucial component of ScottishGlen's HR and payroll infrastructure. This component directly interacts with the Kerberos network authentication system, the ASP.NET web frontend for HR staff, the staff mobile application, and the payroll system's Windows GUI frontend. It is a central piece of the system's architecture and compromise could easily lead to data breaches of highly sensitive information such as home addresses, contact information, and banking details. It could also be exploited for unauthorized access to system accounts, as well as unwanted deletion, modification and insertion of stored data. According to Stuart Skelly, the average GDPR fine in the UK during 2024 was £153,722, making the investment into infrastructure security entirely worthwhile (2025).

An important aspect of PostgreSQL is PL/pgSQL which is defined by its own documentation as "a loadable procedural language for the PostgreSQL database system", (PostgreSQL, n.d.). Essentially, this allows loadable modules to be developed and executed to carry out standardized queries. It is stated that all components of the systems infrastructure have been highly modified by ScottishGlen's development team, in terms of the PostgreSQL database server, the development team has likely implemented custom PL/pgSQL functions to carry out database queries for HR tasks, payroll logic, and other database components of the system. While PL/pgSQL provides important functionality of PostgreSQL for customized development, it can be a huge security risk when used to create dynamic SQL construction functions without proper security considerations.

Dynamic SQL construction is when an SQL query is built by concatenating strings at runtime. It is not inherently bad, it simply requires more care to be taken when practiced. Without proper precautions, the database is vulnerable to SQL injection (CWE-89) which allows attackers to dump or modify the contents of it. Additionally, there are many cases of externally developed PostgreSQL APIs adding in functions vulnerable to SQL injection, which are then imported into previously secure systems. These vulnerabilities are often swiftly patched but remain issues in implemented code due to the frequent occurrence of developers using out of date dependencies and libraries. Furthermore, according to Mohammed Khalil, the majority of vulnerabilities documented in 2025 are injection related (including SQL injection) and have had known solutions since the early 2000s, (2025). This can be seen in *figure 1* below.

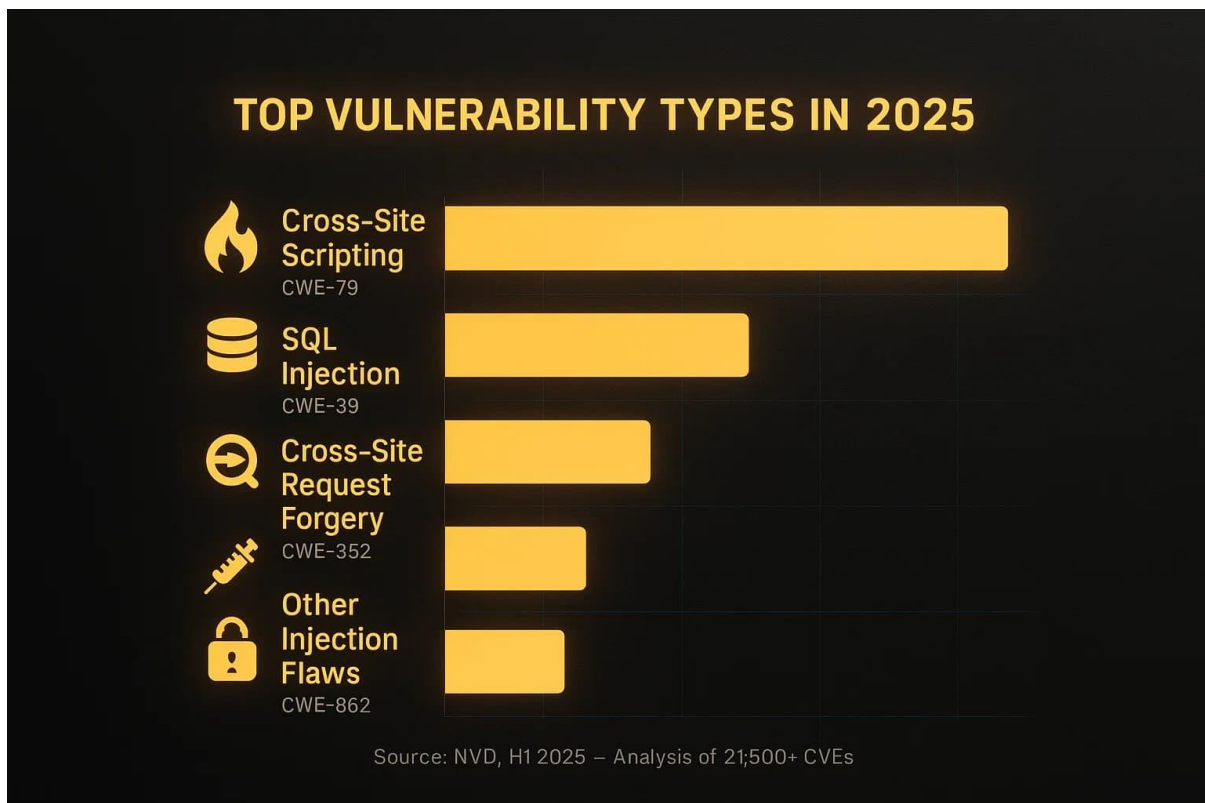


Figure 1. Top Vulnerability Types (Khalil, 2025)

2. Recommendation

Recommendation Overview

While ScottishGlen could currently be at risk of an attack through SQL injection of the PostgreSQL database, the risk can be minimised using secure coding practices and code review. This is also a strong method of mitigating any other potential vulnerabilities present within ScottishGlens systems which involve a software-based attack vector. One of the main benefits of practicing secure coding is that it contributes to catching vulnerabilities in the development process and therefore allowing them to be remediated before attackers have a chance exploited them. The recommended secure coding practice solution involves three components:

- Secure Coding Checklist: A set of rules which developers must systematically check to see if the written code abides by it.
- Static Analysis of Code: Automated Static Application Security Testing (SAST) tools to review code and look for known vulnerable patterns.
- Vulnerability Testing: Testing any user input field with malicious prompts to ensure they are handled appropriately.

OWASP Secure Coding Practices Checklist (OWASP, n.d.)

ScottishGlen is recommended to incorporate various different aspects of secure coding. There is a large OWASP checklist to fully cover the practice of secure coding, therefore this paper will focus on the practices and techniques which directly impact database security. This checklist should be implemented into the development process to ensure that the code follows the known documented rules which make database code safe. This should take place during active coding, and upon concluding a project.

Automated Static Analysis - Semgrep

Static analysis tools such as Semgrep can be used to automate code review. This should not be used as a substitute for the database security checklist and rather an additional precautionary practice. Static analysis tools search for predetermined patterns of code in software, making them useful when searching for known vulnerable code patterns. Semgrep is an SAST tool which has strong database vulnerability pattern finding functions making it the best choice for the task of securing the PostgreSQL database. The results would need to be carefully reviewed by the developer of the code section being reviewed to check for any flagged issues.

Peer Code Review

Peer code review is a very common method of secure code practice. Developers would be paired and review sections of code written by their partner. The purpose of this secure coding practice is to make of for the possibility of human error. Humans can very easily miss things, therefore, if it is reviewed a multiple times and by different developers, vulnerabilities and issues are more likely to get caught. This technique should not replace code review of ones own work and rather be used in addition to it. The developers could also have the responsibility of reviewing their peer's Semgrep results here to ensure that the results were correctly interpreted.

Vulnerability Testing User Input Fields

Another powerful method of secure coding is extensively testing the code to ensure that it works properly. This consists of testing every aspect of the code to ensure it is not functioning in an unintended way. The goal of this is to find scenarios where the code does not work as intended and patch it before it can be used to exploit the system. This helps to identify vulnerabilities before they can be exploited.

Practicality for ScottishGlen

This proposed plan would likely be an effective and efficient solution that suits the 6-developer team and ScottishGlen. Each developer would need to periodically review their own code to ensure that it does not go against any of the new guidelines. Peer code review would also work out well for an even numbered team, as members could double up and review each others code simultaneously. Utilizing an automated static analysis tool like Semgrep would also ease the workload further as this requires very little effort to carry out.

Effectiveness of Mitigation

The effectiveness of implementing secure coding practices can be measured throughout by its probability of success, and its difficulty of practicing. Humans inevitably make mistakes, therefore, the proposed solution intentionally provides multiple different stages of review where vulnerabilities could be caught in code if present. With each developers simply reviewing their own work as well as their partners in peer code review, the workload is evenly dividing amongst the team – even if they grow or shrink in size. Furthermore, the automated static analysis tools provide an extra layer of security by pointing out any previously caught vulnerabilities within the system. Lastly, the OWASP secure coding practices checklist provides a strong guideline for the developers, which if properly followed, should prevent code from being published with present vulnerabilities.

Critical Analysis

Various alternative approaches were considered for remediating the potential SQL injection vulnerabilities present within the ScottishGlen system. Human security training could have been used to build each of the developer's awareness of the situations where code can be vulnerable. However, this is not ideal for a scalable system and can massively reduce the development

efficiency. Furthermore, this approach relies on the developers to catch every vulnerability themselves, which is unreasonable as human error is inevitable in software development. Vulnerability testing the code was also considered. This approach would involve attempting to find vulnerabilities in the code by penetration testing the database and connected systems and features within the HR web interface and mobile app. This is a strong method of catching vulnerabilities, however, relies on catching them post-development. The recommended approach helps to catch vulnerabilities during the development process, reducing the amount of backtracking required.

3. Implementation

CVE-2025-1708 Overview - <https://www.cve.org/CVERecord?id=CVE-2025-1708>

CVE-2025-1708 is an example of an SQL injection vulnerability within a PostgreSQL database and was found within the “Endress + Hauser” system. While this CVE is not a vulnerability within PostgreSQL itself and rather an issue with the development of the system in which it is present, it is still highly relevant as it represents the vulnerabilities created by poor coding practices. The CVE allows the attacker to dump the entire PostgreSQL database and read its contents, a dangerous outcome for a database containing sensitive information such as the data held by ScottishGlen’s database. SQL injection vulnerabilities arise when proper care is not given to SQL query construction in the software development phase, and this is likely what happened for the “Endress + Hauser” system. In SQL queries involving user input, dynamic query construction is utilized. Software developers often expect the end-user to use these input fields as intended, whereas in the real-world, this is commonly not the case. To prevent this vulnerability, input fields require extensive care and must be prepared to handle malicious data.

Vulnerable PostgreSQL Function Demonstration

A small PostgreSQL database and vulnerable function was developed to demonstrate the vulnerabilities in poor SQL development (*figure 2*). The table contains the fields “username” and “password” and contains 3 records of data. The function uses an EXECUTE statement and dynamically constructs itself by appending the user’s input onto the end of the query.



```
GNU nano 7.2 /tmp/vulnerable_database.sql
CREATE DATABASE hr_database;
\c hr_database

CREATE TABLE employees (
    username TEXT,
    password TEXT
);
INSERT INTO employees VALUES ('johndoe', 'secretpass1'), ('sarahdoe', 'anotherpassword'), ('glorpzorp', 'alienpassword');

CREATE FUNCTION vulnerable_func(u TEXT)
RETURNS TABLE(username TEXT, password TEXT) AS $$
BEGIN
    RETURN QUERY EXECUTE 'SELECT username, password FROM employees WHERE username = ' || u || ' ';
END;
$$ LANGUAGE plpgsql;
```

Figure 2. Vulnerable function (1/2)

The function was exploited using the most basic form of SQL injection utilizing an OR statement and then giving a parameter that will always be true, such as “OR 1 = 1”, (*figure 3*).

```
(kali@kali)-[~]
$ sudo nano /tmp/vulnerable_database.sql

(kali@kali)-[~]
$ sudo -u postgres psql hr_database
psql (18.1 (Debian 18.1-2))
Type "help" for help.

hr_database=# SELECT * FROM vulnerable_func ('johndoe');
username | password
-----+-----
johndoe  | secretpass1
johndoe  | secretpass1
(2 rows)

hr_database=# SELECT * FROM vulnerable_func ('' OR ''1''=''1');
username | password
-----+-----
johndoe  | secretpass1
sarahdoe | anotherpassword
glorpzorp | alienpassword
johndoe  | secretpass1
sarahdoe | anotherpassword
glorpzorp | alienpassword
(6 rows)

hr_database=#
```

Figure 3. Vulnerable function (2/2)

OWASP Secure Coding Practices Checklist

The OWASP Secure Coding Practices Checklist should be integrated into the software development and testing lifecycle. In the context of mitigating the SQL injection vulnerabilities in the PostgreSQL database, there are a few areas requiring greater attention outlined within this checklist, such as: input validation, output encoding, cryptographic practices, error handling and logging, database security and general coding practices.

Input validation and output encoding are the primary secure coding practices for preventing SQL injection. User input be carefully analysed by the system, ensuring it does not contain a larger key space than required, as well as preventing unnecessary special characters which are used within programming languages. One method of ensuring SQL security is to theorize and craft all the expected and intended queries the system will need, and then prevent other options, rather than attempting to catch every possible individual issue. Furthermore, all input validation, output encoding, and error handling should be carried out server-side.

Semgrep Static Analysis and Peer Code Review

Semgrep should be used to scan code for vulnerabilities. Semgrep finds vulnerable code patterns using large amounts of previous data. This would immediately flag the vulnerable function shown in *figure 2*. This tool should be used before publishing any code so that vulnerabilities can be patched before they can be exploited.

Additionally, the development team should split into small groups and have the responsibility to review their group's code. The more time code is reviewed, the more likely vulnerabilities are caught. Having both automated tool analysis and manual human analysis increases the possibility of vulnerabilities being caught which may have been missed by the other method.

Input Field Vulnerability Testing

Finally, input fields should be manually tested with various SQL injection prompts to ensure that the implemented vulnerability prevention methods have worked in securing the database. If any of these simulated attacks succeed, then they help identify critical issues with the database and can be used as reference of what to patch. Standard SQL injection, as displayed in *figure 3*, should be the first test. This can be used to view the contents of the database which is already a huge problem. Furthermore, this same method can be used to carry out other types of SQL queries. The same query could be slightly modified to DROP the database instead of breaching its contents (*figure 4*), causing all the data within to be deleted.

```
SELECT * FROM secure_func(''; DROP TABLE employees; --');
```

Figure 4. DROP SQL Query

Time-based SQL injection in the context of PostgreSQL involves the `pg_sleep()` function. By editing the query to contain `''OR pg_sleep(5)'`, attackers can observe whether forms of SQL injection may be possible (*figure 5*). If the database waits 5 seconds to react to the query, then it means that the SQL statement was successfully injected and carried out by the system. While this is primarily an enumeration technique, it can be a critical vulnerability when the system is designed to not display error-output. Time-based SQL injection bypasses this and reassures attackers that they may have an entry point.

```
SELECT * FROM secure_func('' OR pg_sleep(5)--');
```

Figure 5. Time-based SQL Query

Error-based SQL injection is another variant of CWE-89 worth testing for as it is extremely dangerous in situations where the interface does display error-output. This technique has both enumeration and attacking capabilities. A query like the one seen in *figure 6* can be used to leak the version of PostgreSQL which is being used. Attackers could then use this to carry out known exploits for that specific version.

```
SELECT * FROM vulnerable_func('' AND 1=CAST((SELECT version()) AS INT)--');
```

Figure 6. Error-based Enumeration SQL Query

When Error-based SQL injection is used for exploiting the database, the query would instead look more like *figure 7*. If successful, this would return the password for the account in the first record, which is often the administrator account, in an error. The query could be edited from “password” to “username” to acquire the username of this account.

```
SELECT * FROM vulnerable_func('' AND 1=CAST(
  (SELECT password FROM employees LIMIT 1) AS INT
)--');
```

Figure 7. Error-based Exploitation SQL Query

References

- Khalil, M. (2025, October 8). *Vulnerabilities Statistics 2025: Record CVEs, Zero-Days & Exploits*. Retrieved from Deepstrike: <https://deepstrike.io/blog/vulnerability-statistics-2025>
- OWASP. (n.d.). *OWASP Secure Coding Practices - Quick Reference Guide*. Retrieved from OWASP: <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/stable-en/02-checklist/05-checklist>
- Ponemon Institute. (2014). *The SQL Injection Threat Study*. Retrieved from <https://www.ponemon.org/local/upload/file/DB%20Networks%20Research%20Report%20FINAL5.pdf>
- PostgreSQL. (n.d.). *Chapter 41. PL/pgSQL — SQL Procedural Language*. Retrieved from PostgreSQL Documentation Website: <https://www.postgresql.org/docs/current/plpgsql-overview.html>
- Skelly, S. (2025, February 5). *Analysis of Fines Imposed by the Information Commissioner's Office in 2024*. Retrieved from urmconsulting: <https://www.urmconsulting.com/blog/analysis-of-fines-imposed-by-the-information-commissioners-office-in-2024>